Lecture 1
Probability and Bayesian Statistics

From Bayesian statistics to probabilistic machine learning

Probabilistic Machine Learning 2027-1

2026-09-23

Section 1

A familiar modeling language

Probablistic models update based on data

The Beta-Binomial model

Observed

the coin tosses $y_1, \ldots, y_N$

Unknown

the coin bias θ

- A statistical model has data and unknown parameters

The Beta-Binomial model

Observed

the coin tosses $y_1, ..., y_N$

Unknown

the coin bias θ

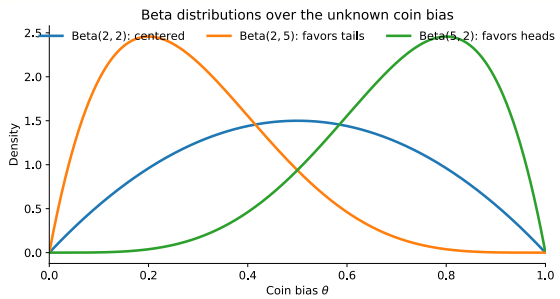
$$\theta \sim \text{Beta}(\alpha, \beta)$$

$$y_i \sim \text{Bernoulli}(\theta), \quad i = 1, ..., N$$

- A statistical model has data and unknown parameters
- How could observed data (y) could have been generated.
- What does it tell us about the parameters (θ)

Models are built out of distributions

$$\theta \sim \text{Beta}(\alpha, \beta)$$

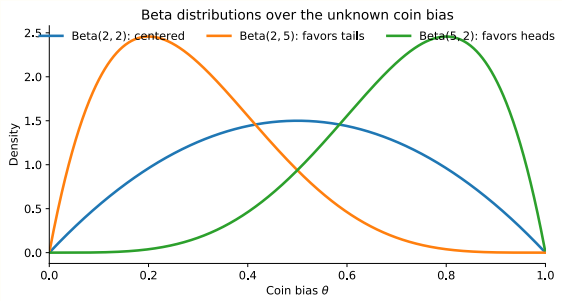


figures/coin_model__beta_distribution.svg

- Beta for continuous variables on $[0, 1]$.

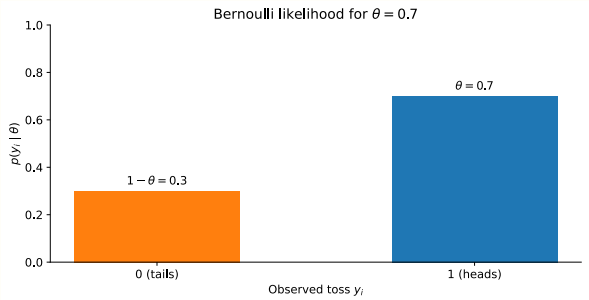
Models are built out of distributions

$$\theta \sim \text{Beta}(\alpha, \beta)$$



figures/coin_model__beta_distribution.svg

$$y_i \sim \text{Bernoulli}(\theta)$$
$$i = 1, \ldots, N$$

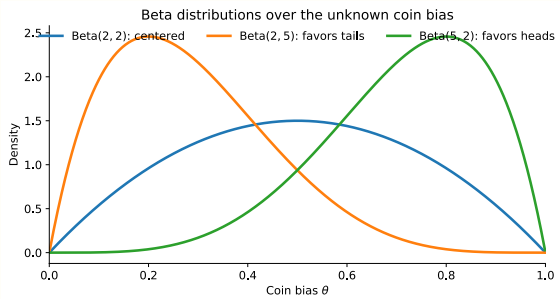


figures/coin_model__bernoulli_likelihood.svg

- Beta for continuous variables on $[0, 1]$.
- Bernoulli four the number of counts in a yes/no process.

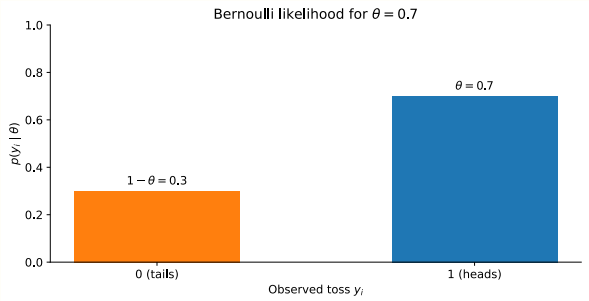
Models are built out of distributions

$$\theta \sim \text{Beta}(\alpha, \beta)$$



figures/coin_model__beta_distribution.svg

$$y_i \sim \text{Bernoulli}(\theta)$$
$$i = 1, ..., N$$



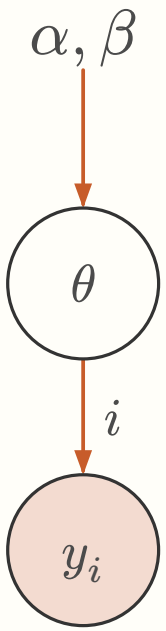
figures/coin_model__bernoulli_likelihood.svg

- Beta for continuous variables on $[0, 1]$.
- Bernoulli four the number of counts in a yes/no process.
- Distributions are parameterized families

Models can be drawn as graphs

$$\theta \sim \text{Beta}(\alpha, \beta)$$

$$y_i \sim \text{Bernoulli}(\theta)$$

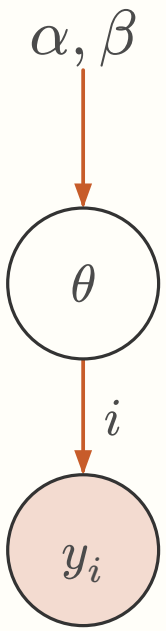


- Nodes represent quantities.
  - Shading marks observed data.

Models can be drawn as graphs

$$\theta \sim \text{Beta}(\alpha, \beta)$$

$$y_i \sim \text{Bernoulli}(\theta)$$

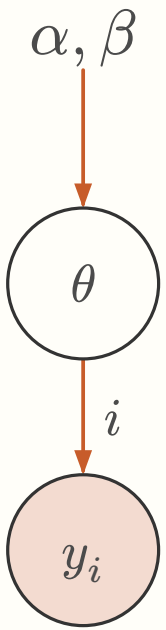


- Nodes represent quantities.
  - Shading marks observed data.
- Arrows encode conditional dependence.
  - We will not use Kruschke diagrams

Models can be drawn as graphs

$$\theta \sim \text{Beta}(\alpha, \beta)$$

$$y_i \sim \text{Bernoulli}(\theta)$$



- Nodes represent quantities.
  - Shading marks observed data.
- Arrows encode conditional dependence.
  - We will not use Kruschke diagrams

In this course, graphs will allow us to factor complex problems into smaller, more tractable problems

Bayes' rule combines prior and likelihood

Model

$$\theta \sim \text{Beta}(\alpha, \beta)$$

$$y_i \sim \text{Bernoulli}(\theta)$$

Bayes' rule combines prior and likelihood

Model

$$\theta \sim \text{Beta}(\alpha, \beta)$$

$$y_i \sim \text{Bernoulli}(\theta)$$

Bayes' rule

$$p(\theta \mid y) = \frac{p(y \mid \theta)p(\theta)}{p(y)}$$

$$\propto p(y \mid \theta)p(\theta)$$

Bayes' rule combines prior and likelihood

Model

$$\theta \sim \text{Beta}(\alpha, \beta)$$

$$y_i \sim \text{Bernoulli}(\theta)$$

Bayes' rule

$$p(\theta \mid y) = \frac{p(y \mid \theta)p(\theta)}{p(y)}$$

$$\propto p(y \mid \theta)p(\theta)$$

- **Prior:** $p(\theta)$
- **Likelihood:** $p(y \mid \theta)$
- **Posterior:** $p(\theta \mid y)$

Bayes' rule combines prior and likelihood

Model

$$\theta \sim \text{Beta}(\alpha, \beta)$$

$$y_i \sim \text{Bernoulli}(\theta)$$

Bayes' rule

$$p(\theta \mid y) = \frac{p(y \mid \theta)p(\theta)}{p(y)}$$

$$\propto p(y \mid \theta)p(\theta)$$

- **Prior:** $p(\theta)$
- **Likelihood:** $p(y \mid \theta)$
- **Posterior:** $p(\theta \mid y)$
- We explored the posterior of θ
  - Treating $p(y)$ as a normalizing constant

Conjugacy permits simple calculations

Model

$$\theta \sim \text{Beta}(\alpha, \beta)$$

$$y_i \sim \text{Bernoulli}(\theta)$$

Bayes rule

$$p(\theta \mid y) \propto p(y \mid \theta)p(\theta)$$

Applying Bayes' rule to our model

Conjugacy permits simple calculations

Model

$$\theta \sim \text{Beta}(\alpha, \beta)$$

$$y_i \sim \text{Bernoulli}(\theta)$$

Bayes rule

$$p(\theta \mid y) \propto p(y \mid \theta) p(\theta)$$

Applying Bayes' rule to our model

Prior $p(\theta) = \text{Beta}(\alpha, \beta)$

Conjugacy permits simple calculations

Model

$$\theta \sim \text{Beta}(\alpha, \beta)$$

$$y_i \sim \text{Bernoulli}(\theta)$$

Bayes rule

$$p(\theta \mid y) \propto p(y \mid \theta)p(\theta)$$

Applying Bayes' rule to our model

Prior $p(\theta) = \text{Beta}(\alpha, \beta)$

Likelihood $p(y \mid \theta) = \text{Binomial}(N, \theta)$

Conjugacy permits simple calculations

Model

$$\theta \sim \text{Beta}(\alpha, \beta)$$

$$y_i \sim \text{Bernoulli}(\theta)$$

Bayes rule

$$p(\theta \mid y) \propto p(y \mid \theta) p(\theta)$$

Applying Bayes' rule to our model

Prior $p(\theta) = \text{Beta}(\alpha, \beta)$

Likelihood $p(y \mid \theta) = \text{Binomial}(N, \theta)$

Conjugacy

The prior and likelihood are conjugate when the posterior belongs to the same distribution family as the prior.

Conjugacy permits simple calculations

Model

$$\theta \sim \text{Beta}(\alpha, \beta)$$

$$y_i \sim \text{Bernoulli}(\theta)$$

Bayes rule

$$p(\theta \mid y) \propto p(y \mid \theta)p(\theta)$$

Applying Bayes' rule to our model

Prior $p(\theta) = \text{Beta}(\alpha, \beta)$

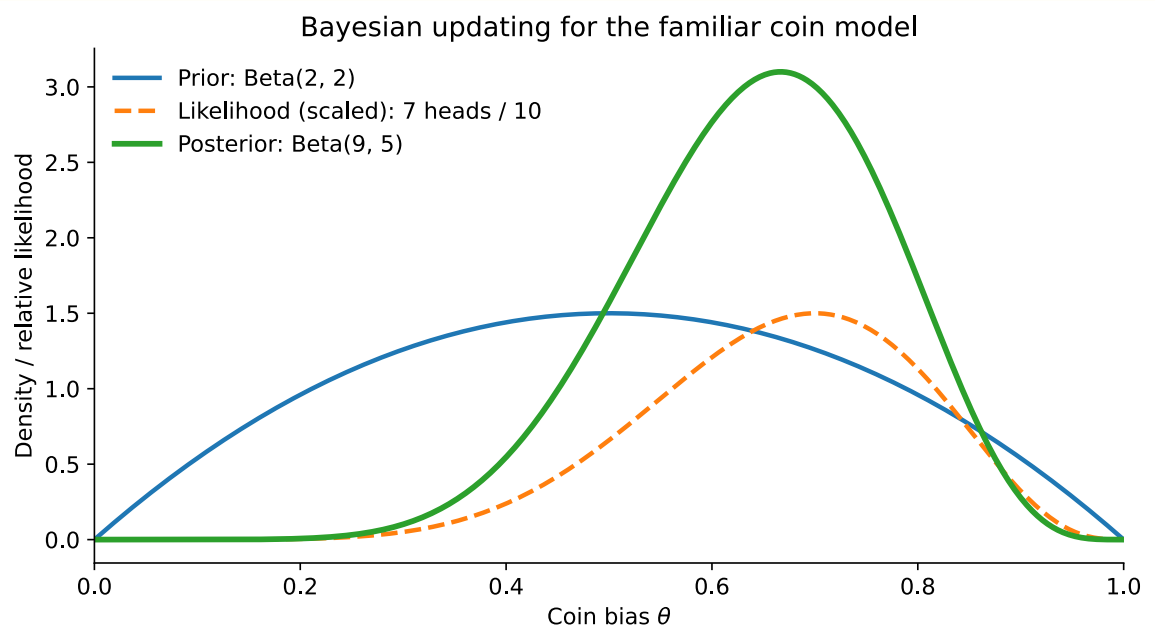
Likelihood $p(y \mid \theta) = \text{Binomial}(N, \theta)$

Posterior $p(\theta \mid y) = \text{Beta}(\alpha + y, \beta + N - y)$

Conjugacy

The prior and likelihood are conjugate when the posterior belongs to the same distribution family as the prior.

The posterior is a distribution representing uncertainty



Bayesian updating for the familiar coin model

`figures/coin_model__prior_likelihood_posterior.svg`

- The prior and the posterior are both probability distributions
- Their width expresses the uncertainty
- The uncertainty in the posterior is less than in the prior

Summary: A familiar modeling language

- Probabilistic models link observed data to parameters
- Graphs show model variables and dependencies
- Bayes' rule combines prior and likelihood into a posterior
- Conjugacy makes some updates analytic
- Posterior width represents remaining uncertainty
- Posterior uncertainty narrows with more data

Section 2

From statistics to machine-learning problems

Same probabilistic language, different organizing questions

Statistics focuses on the unknown parameters

$$p(\theta \mid D) = \frac{p(D \mid \theta)p(\theta)}{p(D)}$$

- Get data → build a model → apply Bayes' rule → interpret θ
  - For many scientific questions, that posterior **is** the answer

Statistics focuses on the unknown parameters

$$p(\theta \mid D) = \frac{p(D \mid \theta)p(\theta)}{p(D)}$$

- Get data → build a model → apply Bayes' rule → interpret θ
  - For many scientific questions, that posterior **is** the answer
- Statistics arose from an interest in the underlying parameters in nature

Machine learning grew out of computer science



- A computational task is an **input/output contract**
  - Produce a function f that maps input x to output y

Machine learning grew out of computer science



- A computational task is an **input/output contract**
  - Produce a function f that maps input x to output y
  - Implementation is secondary to the requirement
  - Multiple implementations may be possible

Machine learning grew out of computer science



- A computational task is an **input/output contract**
  - Produce a function f that maps input x to output y
  - Implementation is secondary to the requirement
  - Multiple implementations may be possible
- Machine learning developed in this context

Early machine learning grew out of classification problems

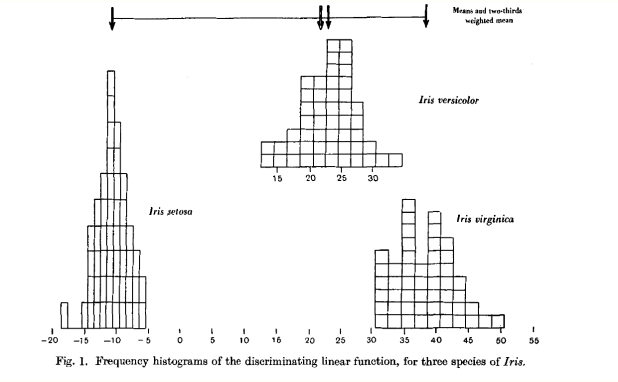


Fig. 1. Frequency histograms of the discriminating linear function, for three species of *Iris*.

Fisher (1936), Fig. 1, The Use of Multiple Measurements in Taxonomic Problems

Every flower started as four numbers. Fisher learned one weighted combination of those four numbers so that species became as separated as possible. The histogram shows the flowers after that learned projection.

Early machine learning grew out of classification problems

Rosenblatt considered the perceptron a model of the vertebrate visual system. He showed how neuron-like units could learn to "recognize" classes of input patterns from experience.

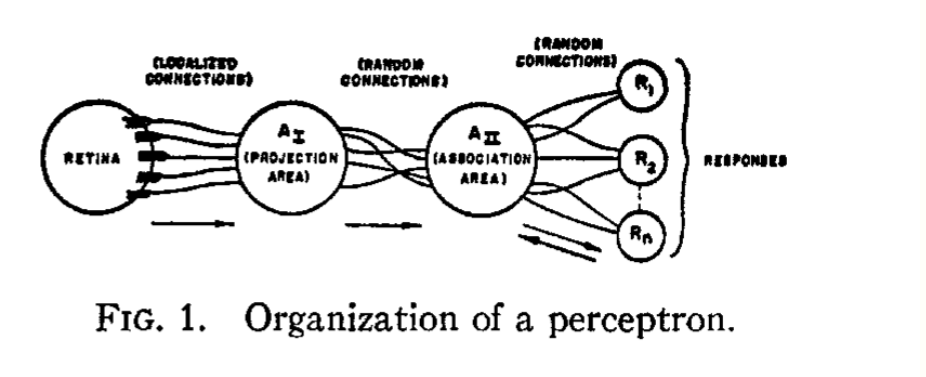


FIG. 1. Organization of a perceptron.

Rosenblatt (1958), Fig. 1, The Perceptron

Early machine learning grew out of classification problems

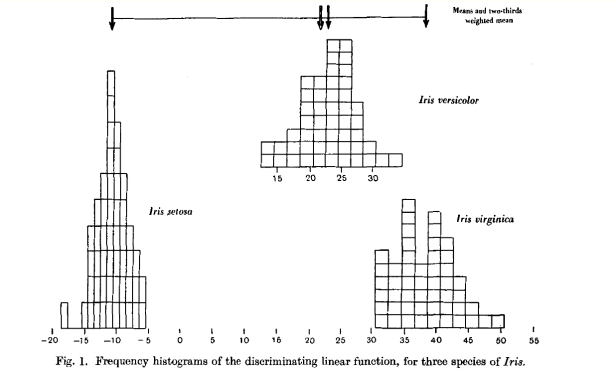


Fig. 1. Frequency histograms of the discriminating linear function, for three species of *Iris*.

Fisher (1936), Fig. 1, The Use of Multiple Measurements in Taxonomic Problems

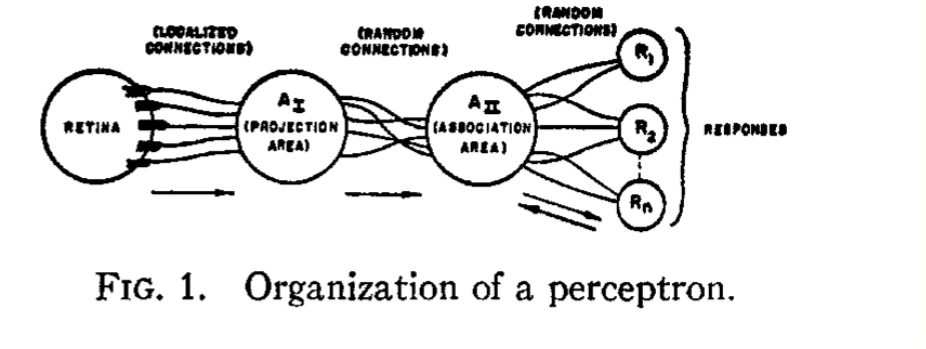
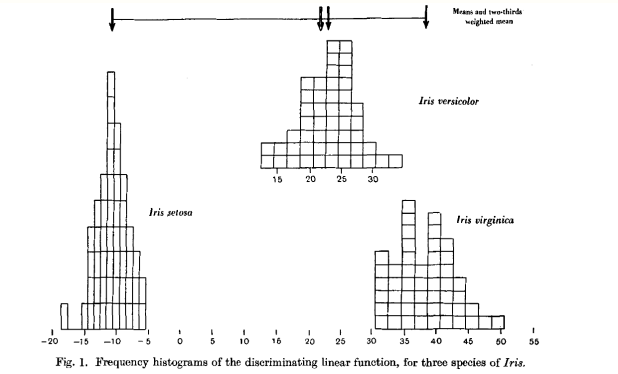


FIG. 1. Organization of a perceptron.

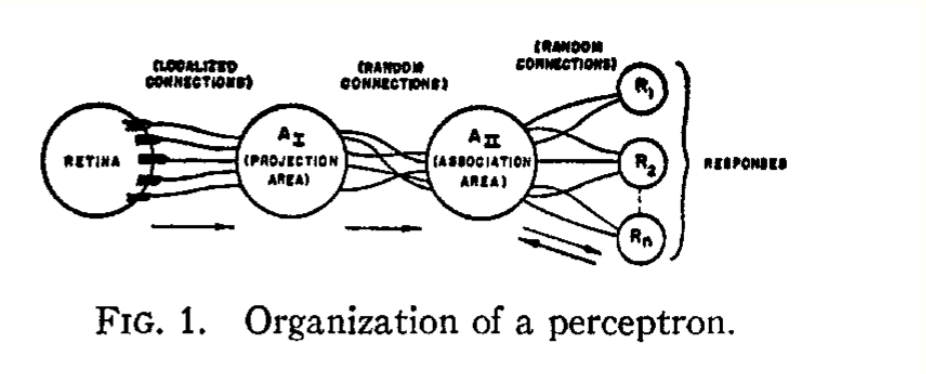
Rosenblatt (1958), Fig. 1, The Perceptron

- Fisher's discriminant and Rosenblatt's perceptron share an interface
  - complex input → class or response

Early machine learning grew out of classification problems



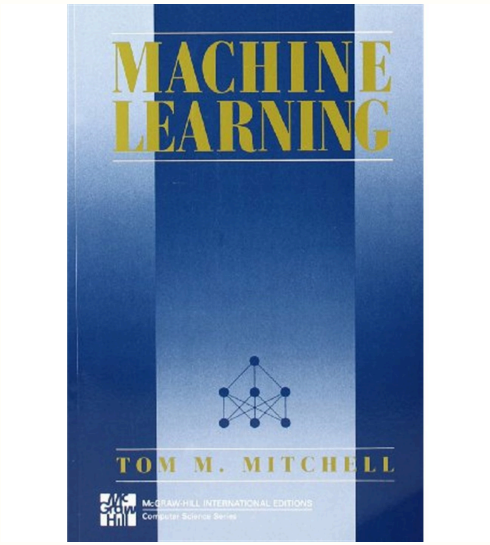
Fisher (1936), Fig. 1, The Use of Multiple Measurements in Taxonomic Problems



Rosenblatt (1958), Fig. 1, The Perceptron

- Fisher's discriminant and Rosenblatt's perceptron share an interface
  - complex input → class or response
- Machine learning asks how data can produce the required function f

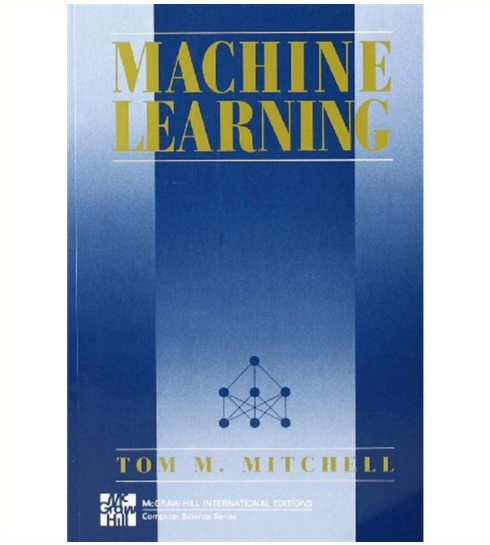
Machine learning defined



Tom M. Mitchell, Machine Learning (1997), cover

A computer program is said to **learn** from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E. ,

Machine learning defined



Tom M. Mitchell, Machine Learning (1997), cover

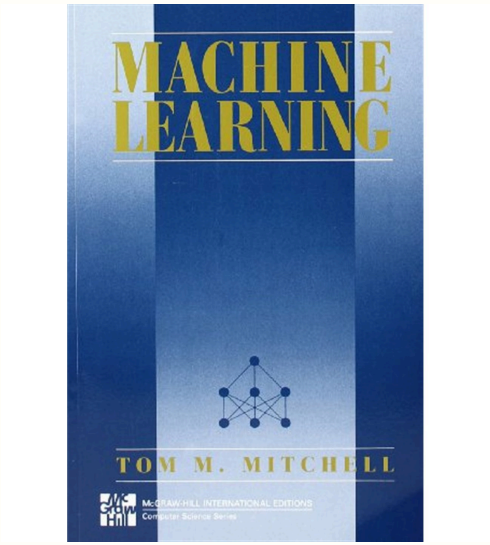
A computer program is said to **learn** from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E. ,

Training

$$f_T : \mathrm{E} \quad \rightarrow \quad f_A$$

E: Experience

Machine learning defined



Tom M. Mitchell, Machine Learning (1997), cover

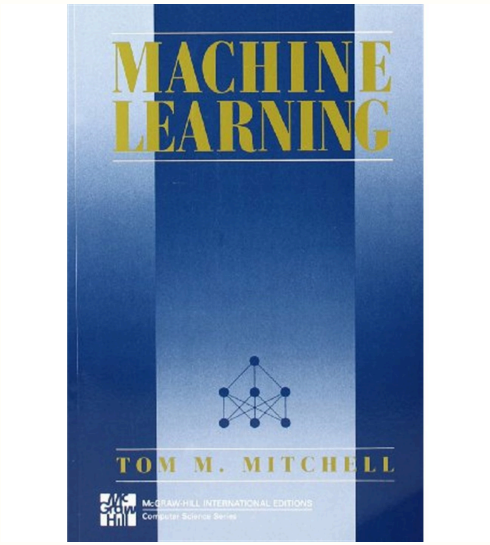
A computer program is said to **learn** from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E. ,

Training **Task**

$f_T : \mathrm{E} \rightarrow f_A$ $f_A : \mathrm{In} \rightarrow \mathrm{Out}$

E: Experience f_A: Task function

Machine learning defined

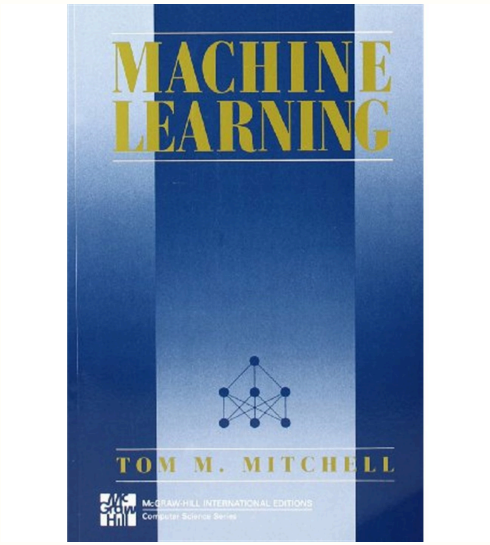


Tom M. Mitchell, Machine Learning (1997), cover

A computer program is said to **learn** from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E. ,

Training	**Task**	**Evaluation**
$f_T : \mathrm{E} \rightarrow f_A$	$f_A : \mathrm{In} \rightarrow \mathrm{Out}$	$f_P : (\mathrm{In}, \mathrm{Out}) \rightarrow \mathrm{P}$
E: Experience	f_A: Task function	P: Performance

Machine learning defined



Tom M. Mitchell, Machine Learning (1997), cover

A computer program is said to **learn** from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E. ,

Training	**Task**	**Evaluation**
$f_T : \mathrm{E} \rightarrow f_A$	$f_A : \mathrm{In} \rightarrow \mathrm{Out}$	$f_P : (\mathrm{In}, \mathrm{Out}) \rightarrow \mathrm{P}$
E: Experience	f_A: Task function	P: Performance

- Machine learning is a mapping from $(E, f_P) \rightarrow f_T \rightarrow f_A$
  - That maximizes f_A performance according to P

Supervised experience supports prediction

Experience: paired examples

$$f_T : \mathrm{E} \quad \to \quad f_A$$

$$f_T : D = \{(x_i, y_i)\}_{i=1}^{N} \quad \to \quad f_D$$

paired examples → learned predictor

Task: prediction

$$f_A : \mathrm{In} \quad \to \quad \mathrm{Out}$$

$$f_D : x_* \quad \to \quad \hat{y}_*$$

new input → predicted output

Supervised experience supports prediction

Experience: paired examples

$$f_T : \mathrm{E} \quad \to \quad f_A$$

$$f_T : D = \{(x_i, y_i)\}_{i=1}^{N} \quad \to \quad f_D$$

paired examples → learned predictor

Task: prediction

$$f_A : \mathrm{In} \quad \to \quad \mathrm{Out}$$

$$f_D : x_* \quad \to \quad \hat{y}_*$$

new input → predicted output

- Experience is a training set $D = \{(x_n, y_n)\}$
  - x: called feature, covariate, or predictor
  - y: called label, target, or response
- Task is to produce a predictor f_D that maps new x_* to correct y_*
- Performance is measured by a loss, introduced shortly

Classification: Iris flowers


Setosa


Versicolor


Virginica

Table 1.1: subset of the Iris design matrix

index	sl	sw	pl	pw	label
0	5.1	3.5	1.4	0.2	Setosa
1	4.9	3.0	1.4	0.2	Setosa
dots	dots	dots	dots	dots	dots
50	7.0	3.2	4.7	1.4	Versicolor
dots	dots	dots	dots	dots	dots
149	5.9	3.0	5.1	1.8	Virginica

sl: sepal length; sw: sepal width; pl: petal length; pw: petal width

Murphy (2025), Book 1, Fig. 1.1 and Table 1.1

- Classification: $y \in \{1, ..., C\}$
  - The classes are unordered and mutually exclusive

Classification: Iris flowers


Setosa


Versicolor


Virginica

Table 1.1: subset of the Iris design matrix

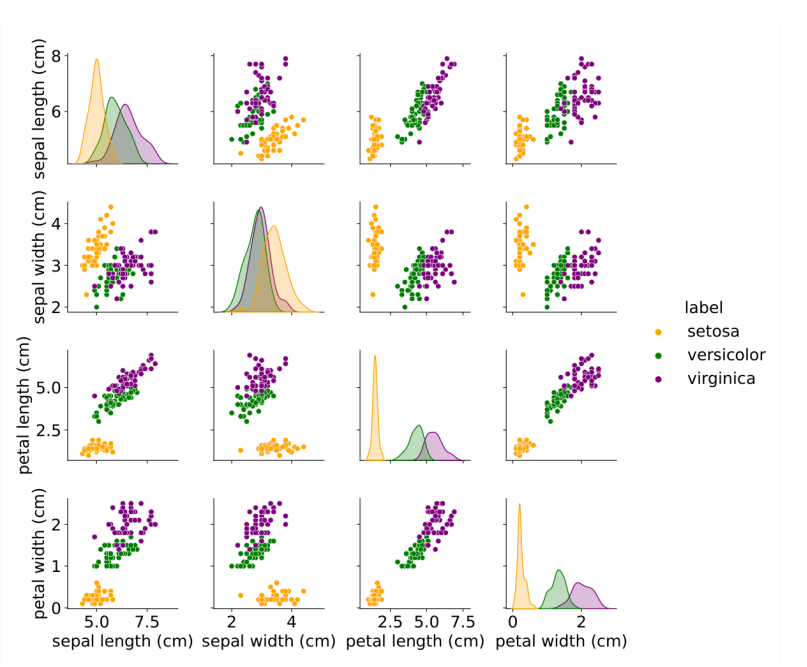
index	sl	sw	pl	pw	label
0	5.1	3.5	1.4	0.2	Setosa
1	4.9	3.0	1.4	0.2	Setosa
dots	dots	dots	dots	dots	dots
50	7.0	3.2	4.7	1.4	Versicolor
dots	dots	dots	dots	dots	dots
149	5.9	3.0	5.1	1.8	Virginica

sl: sepal length; sw: sepal width; pl: petal length; pw: petal width

Murphy (2025), Book 1, Fig. 1.1 and Table 1.1

- Classification: $y \in \{1, ..., C\}$
  - The classes are unordered and mutually exclusive
- For the Iris dataset
  - $y \in \{\text{Setosa}, \text{Versicolor}, \text{Virginica}\}$
  - $x \in \mathbb{R}^4$

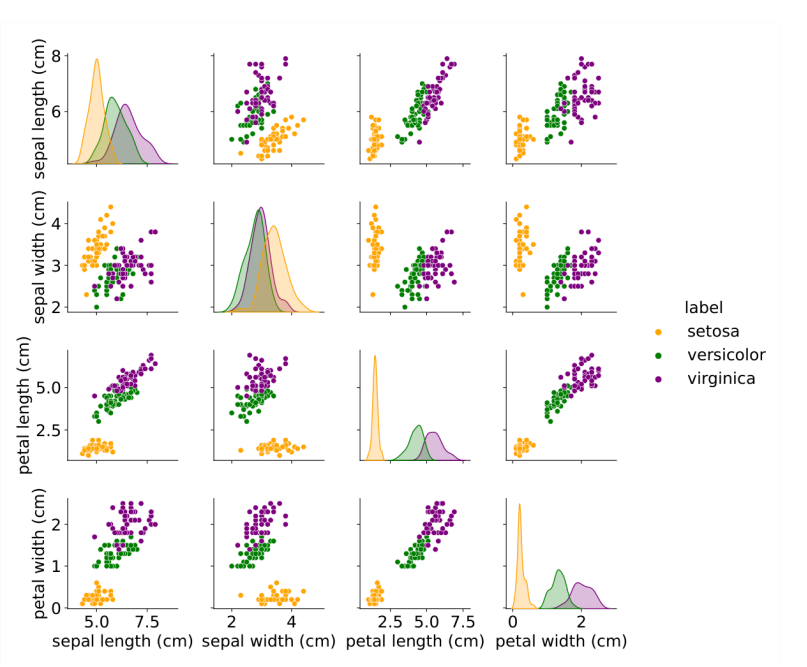
Exploratory data analysis



Murphy (2025), Book 1, Fig. 1.3; generated by iris_plot.ipynb

- Training set
  - 150 labeled examples
  - 50 in each class

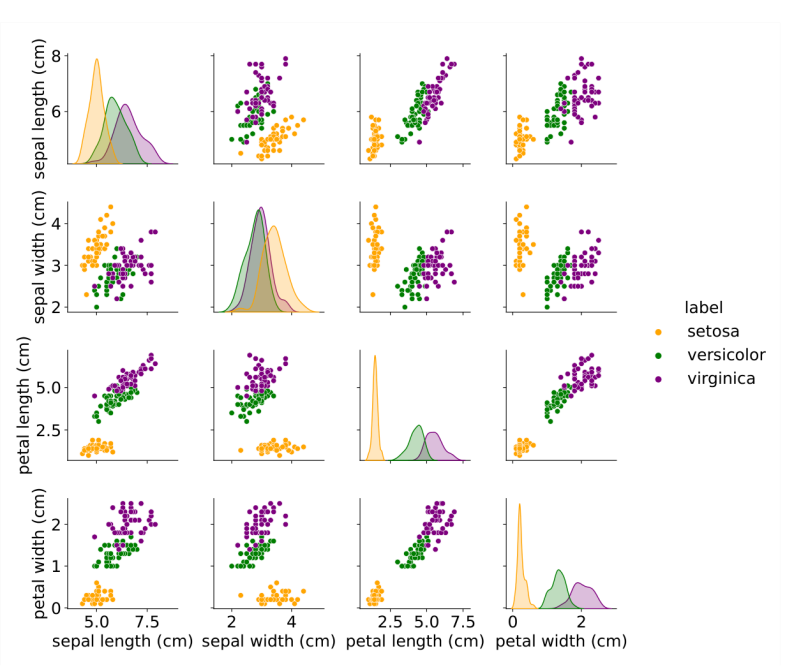
Exploratory data analysis



Murphy (2025), Book 1, Fig. 1.3; generated by iris_plot.ipynb

- Training set
  - 150 labeled examples
  - 50 in each class
- Pair plots shows every pair of features
  - Diagonal shows marginal distributions

Exploratory data analysis



Murphy (2025), Book 1, Fig. 1.3; generated by iris_plot.ipynb

- Training set
  - 150 labeled examples
  - 50 in each class
- Pair plots shows every pair of features
  - Diagonal shows marginal distributions
- Always inspect data before modeling
  - Structure
  - Outliers
  - Missing values

Performance: empirical risk minimization

$$L(x, y; \theta) = \ell(y, f(x; \theta))$$

- The loss function: the cost of predicting $f(x; \theta)$ when the true label is y

Performance: empirical risk minimization

$$L(x, y; \theta) = \ell(y, f(x; \theta))$$

$$L(\theta) = \frac{1}{N} \sum_{n=1}^{N} L(x_n, y_n; \theta)$$

- The loss function: the cost of predicting $f(x; \theta)$ when the true label is y
- The empirical risk: loss evaluated on the training set

Performance: empirical risk minimization

$$L(x, y; \theta) = \ell(y, f(x; \theta))$$

$$L(\theta) = \frac{1}{N} \sum_{n=1}^{N} L(x_n, y_n; \theta)$$

$$\hat{\theta} = \arg\min_{\theta} L(\theta)$$

- The loss function: the cost of predicting $f(x; \theta)$ when the true label is y
- The empirical risk: loss evaluated on the training set
- Empirical risk minimization: minimizing the empirical loss

Performance: empirical risk minimization

$$L(x, y; \theta) = \ell(y, f(x; \theta))$$

$$L(\theta) = \frac{1}{N} \sum_{n=1}^{N} L(x_n, y_n; \theta)$$

$$\hat{\theta} = \arg\min_{\theta} L(\theta)$$

Given samples $D = \{x^1, ..., x^N\}$ from $p(X)$, where $X \in \mathbb{R}$, the empirical distribution is $\hat{p}_{N(x)} = \frac{1}{N} \sum_{n=1}^{N} \delta_{x^n}(x)$.

- The loss function: the cost of predicting $f(x; \theta)$ when the true label is y
- The empirical risk: loss evaluated on the training set
- Empirical risk minimization: minimizing the empirical loss

A loss function

$$L(x, y; \theta) = \ell(y, f(x; \theta))$$

$$\ell(y, f(x; \theta)) = \ell_{01}(y, \hat{y}) = \mathbb{I}(y \neq \hat{y})$$

- A loss function assigns a cost to a prediction when the true label is known

A loss function

$$L(x, y; \theta) = \ell(y, f(x; \theta))$$

$$\ell(y, f(x; \theta)) = \ell_{01}(y, \hat{y}) = \mathbb{I}(y \neq \hat{y})$$

$$\mathbb{I}(e) = \begin{cases} 1 \text{ if } e \text{ is true} \\ 0 \text{ if } e \text{ is false} \end{cases}$$

- A loss function assigns a cost to a prediction when the true label is known
- The zero-one loss is one for a misclassification and zero for a correct classification
- $\mathbb{I}(e)$ is an indicator function: it evaluates whether condition e holds

A probabilistic perspective

$$p(x \mid y = c, \theta)$$

- The exploratory data analysis suggests:
  - Features in each class have similar distributions
  - Different classes have different distributions

A probabilistic perspective

$$p(x \mid y = c, \theta)$$

$$p(y = c \mid x, \theta) \propto p(x \mid y = c, \theta) p(y = c)$$

- The exploratory data analysis suggests:
  - Features in each class have similar distributions
  - Different classes have different distributions
- Bayes' rule converts those ingredients to posterior class probabilities

A probabilistic perspective

$$p(x \mid y = c, \theta)$$

$$p(y = c \mid x, \theta) \propto p(x \mid y = c, \theta) p(y = c)$$

$$\hat{y} = \arg\max_{c \in \mathcal{Y}} p(y = c \mid x; \theta)$$

- The exploratory data analysis suggests:
  - Features in each class have similar distributions
  - Different classes have different distributions
- Bayes' rule converts those ingredients to posterior class probabilities
- A possible classifier: choose the class with maximum a posteriori probability

Task, experience, and performance

Task

What must the learned system do?

$$x \in \mathbb{R}^4 \to \hat{y} \in \mathcal{Y}$$

four measurements → species

- **Task:** predict the Iris species from four sepal and petal measurements

Task, experience, and performance

Task

What must the learned system do?

$$x \in \mathbb{R}^4 \to \hat{y} \in \mathcal{Y}$$

four measurements → species

Experience

What data, feedback, or interaction is available?

$$D = \{(x_n, y_n)\}_{n=1}^{N}$$

150 labeled flowers

- **Task:** predict the Iris species from four sepal and petal measurements
- **Experience:** 150 labeled flowers, with 50 examples from each species

Task, experience, and performance

Task

What must the learned system do?

$$x \in \mathbb{R}^4 \to \hat{y} \in \mathcal{Y}$$

four measurements → species

Experience

What data, feedback, or interaction is available?

$$D = \{(x_n, y_n)\}_{n=1}^{N}$$

150 labeled flowers

Performance

How is success evaluated?

$$L(\theta) = \frac{1}{N} \sum_{n=1}^{N} \mathbb{I}(y_n \neq f(x_n; \theta))$$

zero-one loss / misclassification rate

- **Task:** predict the Iris species from four sepal and petal measurements
- **Experience:** 150 labeled flowers, with 50 examples from each species
- **Performance:** mean zero-one loss, the training misclassification rate

Summary: From statistics to machine learning

- Statistics and computer science
  - Statistics focused on the values of unknown parameters
  - Computer science develops functions that map inputs to outputs

Summary: From statistics to machine learning

- Statistics and computer science
  - Statistics focused on the values of unknown parameters
  - Computer science develops functions that map inputs to outputs
- Machine learning is a branch of computer science
  - Where the function is learned from experience

Summary: From statistics to machine learning

- Statistics and computer science
  - Statistics focused on the values of unknown parameters
  - Computer science develops functions that map inputs to outputs
- Machine learning is a branch of computer science
  - Where the function is learned from experience
- A learning problem is defined by three ingredients: task, experience, and performance
  - One canonical pairing uses supervised experience $D = \{(x_n, y_n)\}$ for prediction
  - The experience regime and the task are related, but they are not the same taxonomy

Section 3

Classes of machine learning problems

Experience regimes, task classes, and performance criteria

Experience taxonomy

Supervised

$$D = \{(x_n, y_n)\}$$

> Experience describes what the learner gets to observe

Experience taxonomy

Supervised

$$D = \{(x_n, y_n)\}$$

Unsupervised

$$D = \{x_n\}$$

> Experience describes what the learner gets to observe

Experience taxonomy

Supervised

$$D = \{(x_n, y_n)\}$$

Unsupervised

$$D = \{x_n\}$$

Self-supervised

$$D = \{x_n, f(x_n)\} \text{ or } \{f(x_n), x_n\}$$

> Experience describes what the learner gets to observe

Experience taxonomy

Supervised

$$D = \{(x_n, y_n)\}$$

Unsupervised

$$D = \{x_n\}$$

Self-supervised

$$D = \{x_n, f(x_n)\} \text{ or } \{f(x_n), x_n\}$$

Reinforcement / interaction

$$\begin{aligned} \text{decision} &: x(t) \to a(t) \\ \text{action} &: (x(t), a(t)) \to x(t+1) \\ \text{reward} &: x(t+1) \to r(t+1) \end{aligned}$$

> Experience describes what the learner gets to observe

Task taxonomy

Prediction

Task: $\{(x_n, y_n)\} \to \hat{y} = f_D(x_*)$

Performance: $f_P(y_*, \hat{y})$

Prediction

Basic task	Map inputs to outputs
Performance	Successful prediction
Example	Classification or regression

Task taxonomy

Prediction

Task: $\{(x_n, y_n)\} \to \hat{y} = f_D(x_*)$

Performance: $f_P(y_*, \hat{y})$

Generation

Task: $\{x_n\} \to (\{c\}, p_D(x \mid c)) \to x_* \sim p_D(x \mid c)$

Performance: $f_P(p(x \mid c), x_*)$

Generation

Basic task	Produce new examples
Performance	Similarity to the true distribution
Example	Text or image synthesis

Task taxonomy

Prediction

Task: $\{(x_n, y_n)\} \to \hat{y} = f_D(x_*)$

Performance: $f_P(y_*, \hat{y})$

Generation

Task: $\{x_n\} \to (\{c\}, p_D(x \mid c)) \to x_* \sim p_D(x \mid c)$

Performance: $f_P(p(x \mid c), x_*)$

Discovery

Task: $\{x_n\} \to (\{z\}, f_D(x)) \to \hat{z} = f_D(x_*)$

Performance: $f_P(\hat{z}, x_*)$

Discovery

Basic task	Find a representation of the data
Performance	Quality of the discovered representation
Example	Clustering or dimensionality reduction

Task taxonomy

Prediction

Task: $\{(x_n, y_n)\} \to \hat{y} = f_D(x_*)$

Performance: $f_P(y_*, \hat{y})$

Generation

Task: $\{x_n\} \to (\{c\}, p_D(x \mid c)) \to x_* \sim p_D(x \mid c)$

Performance: $f_P(p(x \mid c), x_*)$

Discovery

Task: $\{x_n\} \to (\{z\}, f_D(x)) \to \hat{z} = f_D(x_*)$

Performance: $f_P(\hat{z}, x_*)$

Action

Task: $D \to a_* \sim \pi_D(\cdot \mid \{x_{t,*}\}) \to (x_{t+1}, r_{t+1}) = a_t(x_t)$

Performance: $f_P(\{r_t\})$

Action

Basic task	Choose actions to maximize future reward
Performance	Ability to achieve the desired outcome
Example	Reinforcement learning

Task taxonomy

Prediction

Task: $\{(x_n, y_n)\} \to \hat{y} = f_D(x_*)$

Performance: $f_P(y_*, \hat{y})$

Generation

Task: $\{x_n\} \to (\{c\}, p_D(x \mid c)) \to x_* \sim p_D(x \mid c)$

Performance: $f_P(p(x \mid c), x_*)$

Discovery

Task: $\{x_n\} \to (\{z\}, f_D(x)) \to \hat{z} = f_D(x_*)$

Performance: $f_P(\hat{z}, x_*)$

Action

Task: $D \to a_* \sim \pi_D(\cdot \mid \{x_{t,*}\}) \to (x_{t+1}, r_{t+1}) = a_t(x_t)$

Performance: $f_P(\{r_t\})$

Prediction	Map inputs to outputs
Generation	Produce new examples
Discovery	Find a representation of the data
Action	Choose actions to achieve desired outcomes

There are many combinations of experience and task

Experience	Prediction	Generation	Discovery	Action
Supervised	Classification	—	—	Imitation learning
Unsupervised	—	Density modeling	Clustering / PCA	—
Self-supervised	Next-token prediction	Language generation	Representation learning	—
Reinforcement / interaction	—	—	—	Policy learning

- Not every type of experience supports every task.
- Common pairings do exist.

Summary: Two taxonomies, three ingredients

- **Learning problem** = task + experience + performance
- **Experience regimes:** supervised, unsupervised, self-supervised, interactive
- **Task classes:** prediction, generation, discovery, action
- Probabilistic modeling provides a unifying language

Section 4

From datasets to learning problems

The same dataset can pose different learning problems

Popular datasets

Iris





AGE
TASKS

Since 1936 – about 90 years
species prediction; clustering; visualization

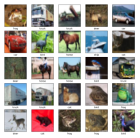
MNIST



AGE
TASKS

Since 1998 – about 28 years
digit classification; representation learning; generation

CIFAR-10



AGE
TASKS

Since 2009 – about 17 years
object classification; robustness; generation

IMDB movie reviews

1. this film was just brilliant casting location scenery story direction everyone's really suited the part they played robert <UNK> is an amazing actor ...
2. big hair big boobs bad music and a giant safety pin these are the words to best describe this terrible movie i love cheesy horror movies and i've seen hundreds...

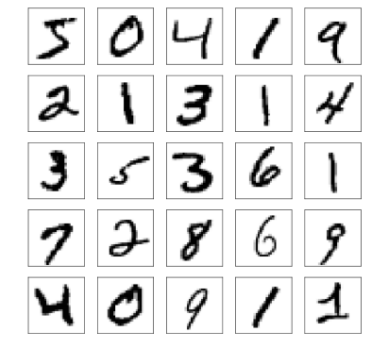
AGE
TASKS

Since 2011 – about 15 years
sentiment classification; text representation

Murphy (2025), Book 1, Figs. 1.1, 1.12a, 1.13b, and Table 1.3

- Machine learning is built around a small number of canonical datasets
  - These four datasets are commonly used
- Each dataset supports multiple tasks
- Datasets support benchmarks that make comparisons easy

MNIST: the same taxonomy, one dataset



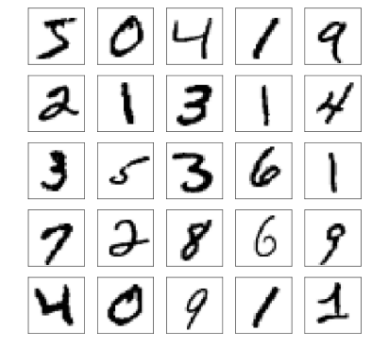
Prediction

$$\hat{y} = f_D(x_*)$$

$$x_n \in \{0, \ldots, 255\}^{28 \times 28}, \quad y_n \in \{0, \ldots, 9\}$$

Murphy (2025), Book 1, Fig. 1.12a

MNIST: the same taxonomy, one dataset



Prediction

$$\hat{y} = f_D(x_*)$$

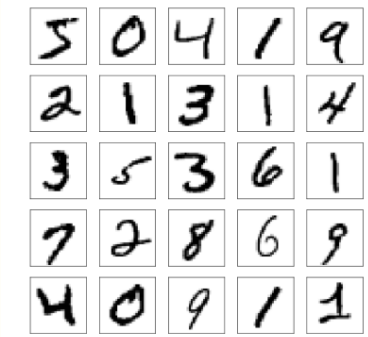
Discovery

$$\hat{z} = f_D(x_*)$$

$$x_n \in \{0, \ldots, 255\}^{28 \times 28}, \quad y_n \in \{0, \ldots, 9\}$$

Murphy (2025), Book 1, Fig. 1.12a

MNIST: the same taxonomy, one dataset



$x_n \in \{0, \ldots, 255\}^{28 \times 28}, \quad y_n \in \{0, \ldots, 9\}$

Prediction

$$\hat{y} = f_D(x_*)$$

Discovery

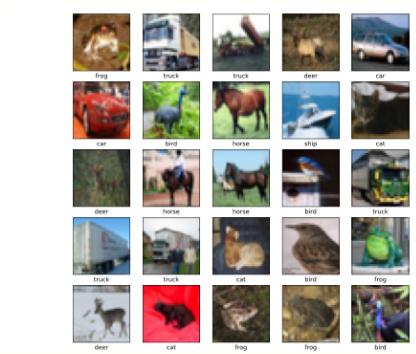
$$\hat{z} = f_D(x_*)$$

Generation

$$x_* \sim p_D(x \mid c)$$

Murphy (2025), Book 1, Fig. 1.12a

CIFAR-10: higher-dimensional inputs, the same questions



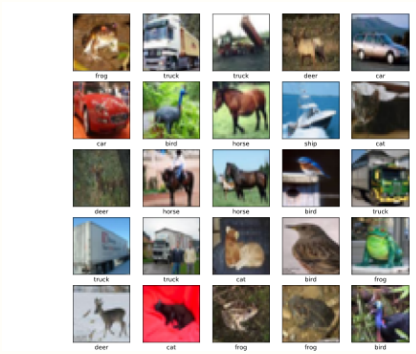
Prediction

$$\hat{y} = f_D(x_*)$$

$$x_n \in \mathbb{R}^{32\times32\times3}, \quad y_n \in \{1, \ldots, 10\}$$

Murphy (2025), Book 1, Fig. 1.13b

CIFAR-10: higher-dimensional inputs, the same questions



Prediction

$$\hat{y} = f_D(x_*)$$

$$L(\theta; D_{\text{train}}) \quad (D_{\text{train}}) \qquad \rightarrow \qquad L(\theta; D_{\text{test}}) \quad (D_{\text{test}})$$

$$x_n \in \mathbb{R}^{32\times32\times3}, \quad y_n \in \{1, \ldots, 10\}$$

Murphy (2025), Book 1, Fig. 1.13b

IMDB reviews: text as sequence data

1. this film was just brilliant casting location scenery story direction everyone's really suited the part they played robert <UNK> is an amazing actor ...
2. big hair big boobs bad music and a giant safety pin these are the words to best describe this terrible movie i love cheesy horror movies and i've seen hundreds...

Prediction

$$\hat{y} = f_D(x_*)$$

Murphy (2025), Book 1, Table 1.3

IMDB reviews: text as sequence data

1. this film was just brilliant casting location scenery story direction everyone's really suited the part they played robert <UNK> is an amazing actor ...
2. big hair big boobs bad music and a giant safety pin these are the words to best describe this terrible movie i love cheesy horror movies and i've seen hundreds...

Prediction

$$\hat{y} = f_D(x_*)$$

Discovery

$$\hat{z} = f_D(x_*)$$

Murphy (2025), Book 1, Table 1.3

IMDB reviews: text as sequence data

1. this film was just brilliant casting location scenery story direction everyone's really suited the part they played robert <UNK> is an amazing actor ...
2. big hair big boobs bad music and a giant safety pin these are the words to best describe this terrible movie i love cheesy horror movies and i've seen hundreds...

Prediction

$$\hat{y} = f_D(x_*)$$

Discovery

$$\hat{z} = f_D(x_*)$$

Generation

$$w_t \sim p_D(w_t \mid w_{<t})$$

Murphy (2025), Book 1, Table 1.3

Learning problems become probabilistic queries

Prediction

Task: $\{(x_n, y_n)\} \to \hat{y} = f_D(x_*)$

Performance: $f_P(y_*, \hat{y})$

Generation

Task: $\{x_n\} \to (\{c\}, p_D(x \mid c)) \to x_* \sim p_D(x \mid c)$

Performance: $f_P(p(x \mid c), x_*)$

Discovery

Task: $\{x_n\} \to (\{z\}, f_D(x)) \to \hat{z} = f_D(x_*)$

Performance: $f_P(\hat{z}, x_*)$

Action

Task: $D \to a_* \sim \pi_D(\cdot \mid \{x_{t,*}\}) \to (x_{t+1}, r_{t+1}) = a_t(x_t)$

Performance: $f_P(\{r_t\})$

- Every task invokes probablity models: $p_D(y|x)$, $p_D(x|c)$, $p_D(z|x)$
- Performances are expectations over future cases: $f_P = E[\cdot]$
- We return to the probablistic modeling tools of Bayesian statistics

Summary: From datasets to learning problems

- A dataset is not a task
- A learning problem names variables and a query
  - prediction: summarize $p_D(y_* \mid x_*)$
  - discovery: infer $p_D(z_* \mid x_*)$
  - generation: sample from $p_D(x \mid c)$

Section 5

Probability for Machine learning

Summaries, conditional summaries, and hidden structure

Machine-learning queries are large and structured

Observed

x_{obs}
features, measurements, context

Latent

z
state, representation, missing value

Output

y_*
prediction, sample, decision

$$p_{D(x_{\text{obs}}, z, y_*)}$$

- Inputs and outputs may be high-dimensional objects
- Relevant quantities may be latent or only partly observed
- Different tasks ask different questions of the same joint model
- Direct calculation with the **full joint distribution** quickly becomes **expensive**

Key operations for simplifying distributions

Summarize
$E[g(Y) \mid X = x]$
prediction or expected performance

Marginalize
$p(x) = \int p(x, z)\, \mathrm{d}z$
remove variables outside the task

Condition
$p(z \mid x)$
infer an unknown from observations

one joint model $p_D(x, y, z)$ $\rightarrow$ many task-specific answers

- Predictions and performance measures often require conditional expectations
- Latent variables must be inferred or averaged out
- Prediction, discovery, generation, and imputation reuse these operations

Structure makes the calculations possible

Full joint

$p(v_1, \ldots, v_m)$
all dependencies represented directly

Dependence

$E[\boldsymbol{x}], \mathrm{Cov}[\boldsymbol{x}]$
multivariate summaries

- Covariance and multivariate models describe dependence among many variables

Structure makes the calculations possible

Full joint

$p(v_1, \ldots, v_m)$
all dependencies represented directly

→

Structured joint

$p(v_1, \ldots, v_m) = \prod_i p\left(v_i \mid v_{\mathrm{pa}(i)}\right)$
conditional independence exposes smaller pieces

Dependence

$E[\boldsymbol{x}], \mathrm{Cov}[\boldsymbol{x}]$
multivariate summaries

- Covariance and multivariate models describe dependence among many variables
- Conditional independence reduces the joint model to local calculations

Structure makes the calculations possible

Full joint

$p(v_1, \ldots, v_m)$
all dependencies represented directly

→

Structured joint

$p(v_1, \ldots, v_m) = \prod_i p\left(v_i \mid v_{\mathrm{pa}(i)}\right)$
conditional independence exposes smaller pieces

Dependence

$E[\boldsymbol{x}], \mathrm{Cov}[\boldsymbol{x}]$
multivariate summaries

Tractable family

$\mathcal{N}(\mu, \Sigma)$
analytic marginals and conditionals

Computation

exact when possible
approximate when necessary

- Covariance and multivariate models describe dependence among many variables
- Conditional independence reduces the joint model to local calculations
- The linear Gaussian system will combine these ideas in one tractable model

Expectation

Continuous

$$E[X] = \int x p(x)\,\mathrm{d}x$$

Discrete

$$E[X] = \sum_x x p(x)$$

$$E[aX + b] = aE[X] + b$$

Course schematic; Book 1 §2.2.5

Linearity of expectation

$X_1, \ldots, X_m$
random quantities

$\rightarrow$

$\sum_i X_i$
$$E\left[\sum_i X_i\right] = \sum_i E[X_i]$$

$$E[aX + b] = aE[X] + b$$

Variance and second moments

Variance

$$V[X] = E[(X-\mu)^2] = E[X^2] - \mu^2$$

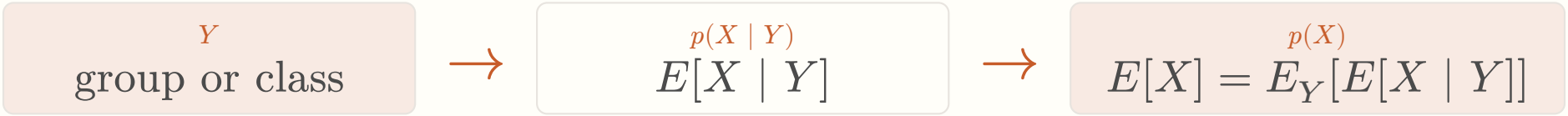
Second moment

$$E[X^2] = \sigma^2 + \mu^2$$

Course schematic; Book 1 §2.2.5

,

Conditional expectation



Course schematic; Book 1 §2.2.5

- We may condition on many different things, not just group
- $p(X \mid Y)$ is a function of X and Y, and $E[X \mid Y]$ is a function of Y

Total variance

$$V[X] = \underbrace{E_Y[V[X \mid Y]]}_{\text{within}} + \underbrace{V_Y[E[X \mid Y]]}_{\text{between}}$$

$V[X \mid Y]$
conditional spread

+

$E[X \mid Y]$
conditional location

Course schematic; Book 1 §2.2.5

Summary: Moments and conditional moments

- Expectation: $E[X]$
- Variance: $V[X] = E[X^2] - E[X]^2$
- Conditional expectation: $E[X \mid Y]$
- Total expectation: $E[X] = E_{Y[E[X \mid Y]]}$
- Total variance: within-condition variation plus between-condition variation

Section 6

Categorical, multinomial, and softmax

The probability language of multiclass labels

Categorical labels

$$\mathrm{Cat}(y \mid \theta) = \prod_{c=1}^{C} \theta_c^{I(y=c)}$$

$y \in \{1, \ldots, C\}$
one label

$\rightarrow$

$\theta \in \Delta^{C-1}$
$0 \leq \theta_c \leq 1$
$\sum_c \theta_c = 1$

One-hot encoding

$y = 2$
class index

$\rightarrow$

$\boldsymbol{y} = (0, 1, 0)^T$
one-hot vector

$$\mathrm{Cat}(y \mid \theta) = \prod_{c=1}^{C} \theta_c^{y_c}$$

Multinomial counts

$$N = (N_1, \ldots, N_C) \quad \sum_c N_c = N \quad \rightarrow \quad \mathrm{Mult}(N \mid N, \theta) \quad \binom{N}{N_1, \ldots, N_C} \prod_{c=1}^{C} \theta_c^{N_c}$$

Course schematic; Book 1 §2.5

Softmax

$a \in \mathbb{R}^C$
logits



Softmax

$a \in \mathbb{R}^C$
logits

→

$\text{softmax}(a)_c = \frac{\exp(a_c)}{\sum_{c'} \exp(a_{c'})}$
normalization

→

Softmax

$a \in \mathbb{R}^C$
logits

→

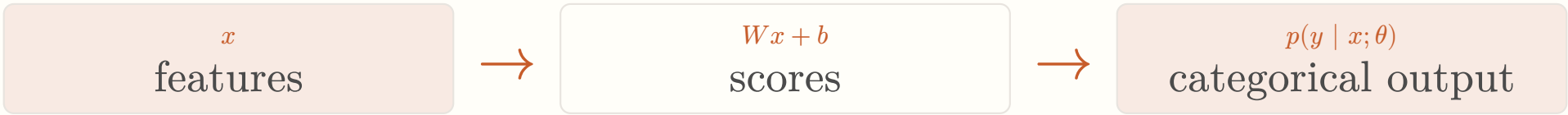
$\text{softmax}(a)_c = \frac{\exp(a_c)}{\sum_{c'} \exp(a_{c'})}$
normalization

→

$\theta \in \Delta^{C-1}$
class probabilities

Multiclass logistic notation

$$p(y \mid x; \theta) = \mathrm{Cat}(y \mid \mathrm{softmax}(Wx + b))$$



Summary: Categorical, multinomial, and softmax

- Categorical: one multiclass label
- One-hot vectors encode class membership
- Multinomial: counts from repeated categorical draws
- Softmax maps logits to the probability simplex
- $p(y \mid x; \theta) = \mathrm{Cat}(y \mid \mathrm{softmax}(Wx + b))$

Section 7

Dirac delta and empirical distribution

Observed data as a distribution of point masses

Point mass

$$\lim_{\sigma \to 0} \mathcal{N}(y \mid \mu, \sigma^2) = \delta(y - \mu)$$

$\mathcal{N}(y \mid \mu, \sigma^2)$
σ^2 tends to 0

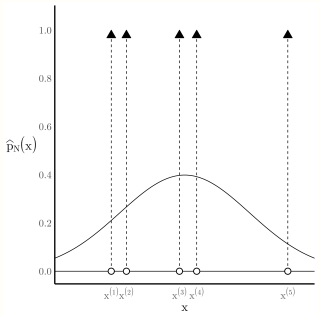
→

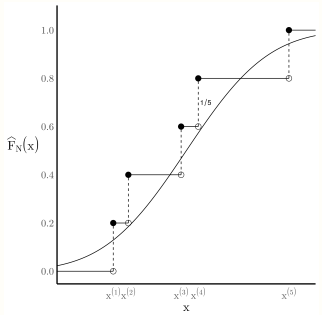
$\delta(y - \mu)$
all mass at μ

Empirical distribution

$$p_{D(y)} = \frac{1}{N} \sum_{n=1}^{N} \delta(y - y_n)$$

The PMF places equal mass on each observation; the CDF accumulates that mass.





Murphy (2025), Book 1, Fig. 2.18A--B; official book figure files

Summary: Empirical distribution

- $\delta(y - \mu)$: a point mass at μ
- $p_{D(y)} = \frac{1}{N} \sum_n \delta(y - y_n)$
- The empirical distribution records observations without smoothing

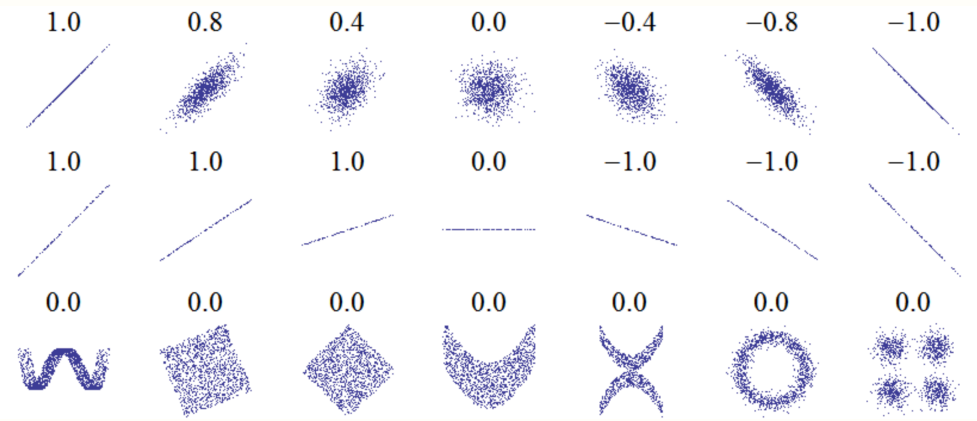
Section 8

Covariance, correlation, and dependence

Joint variation as geometry

Covariance

$$\mathrm{Cov}[X, Y] = E[(X - E[X])(Y - E[Y])]$$



Murphy (2025), Book 1, Fig. 3.1; covariance-pattern gallery

Covariance matrix

$$\mathrm{Cov}[\boldsymbol{x}] = E\big[(\boldsymbol{x} - E[\boldsymbol{x}])(\boldsymbol{x} - E[\boldsymbol{x}])^T\big] = \Sigma$$

$$E\big[\boldsymbol{x}\boldsymbol{x}^T\big] = \Sigma + \mu\mu^T$$

$$\Sigma = \begin{pmatrix} \sigma_1^2 & \mathrm{Cov}[X_1, X_2] \\ \mathrm{Cov}[X_2, X_1] & \sigma_2^2 \end{pmatrix}$$

Affine transformations

x
random vector

→

$Ax + b$
affine map

$$\mathrm{Cov}[A\boldsymbol{x} + b] = A\ \mathrm{Cov}[\boldsymbol{x}]A^T$$

Correlation

$$\rho = \operatorname{corr}[X, Y] = \frac{\operatorname{Cov}[X,Y]}{\sqrt{V[X]V[Y]}}$$

$\rho \approx 1$	$\rho \approx 0$	$\rho \approx -1$
aligned variation	no linear association	opposed variation

Summary: Covariance and dependence

- Covariance: joint variation around the means
- Σ: covariance matrix
- $E[\boldsymbol{x}\boldsymbol{x}^T] = \Sigma + \mu\mu^T$
- $\mathrm{Cov}[A\boldsymbol{x} + b] = A\Sigma A^T$
- Correlation rescales covariance

Section 9

Multivariate Gaussian and conditioning

Geometry, marginalization, and conditioning

Gaussian geometry

$$\mathcal{N}(y \mid \mu, \Sigma) = \frac{1}{(2\pi)^{\frac{D}{2}} \, |\Sigma|^{\frac{1}{2}}} \exp\!\left(-\tfrac{1}{2}(y-\mu)^T \Sigma^{-1}(y-\mu)\right)$$

$\Sigma = \sigma^2 I$

spherical

Gaussian geometry

$$\mathcal{N}(y \mid \mu, \Sigma) = \frac{1}{(2\pi)^{\frac{D}{2}} \, |\Sigma|^{\frac{1}{2}}} \exp\!\left(-\tfrac{1}{2}(y-\mu)^T \Sigma^{-1}(y-\mu)\right)$$

$\Sigma = \sigma^2 I$
spherical

$\Sigma = \mathrm{diag}(\sigma_1^2, \sigma_2^2)$
axis-aligned

Gaussian geometry

$$\mathcal{N}(y \mid \mu, \Sigma) = \frac{1}{(2\pi)^{\frac{D}{2}} \, |\Sigma|^{\frac{1}{2}}} \exp\!\left(-\tfrac{1}{2}(y-\mu)^T \Sigma^{-1} (y-\mu)\right)$$

$\Sigma = \sigma^2 I$	$\Sigma = \mathrm{diag}(\sigma_1^2, \sigma_2^2)$	$\Sigma_{12} \neq 0$
spherical	axis-aligned	tilted

Mahalanobis distance

$$\Delta^2 = (y-\mu)^T \Sigma^{-1} (y-\mu)$$

$y - \mu$
displacement

→

Σ^{-1}
geometry-aware scaling

Typical set

$$\boldsymbol{x} \sim \mathcal{N}(\mathbf{0}, I_D), \quad R^2 = \|\boldsymbol{x}\|^2 = \sum_{i=1}^{D} x_i^2$$

$E[R^2] = D$
radius squared

,

$\frac{\mathrm{sd}(R^2)}{E[R^2]} = \sqrt{\frac{2}{D}}$
relative width

Gaussian blocks and marginals

$$y = \begin{pmatrix} y_1 \\ y_2 \end{pmatrix}, \quad \mu = \begin{pmatrix} \mu_1 \\ \mu_2 \end{pmatrix}, \quad \Sigma = \begin{pmatrix} \Sigma_{11} & \Sigma_{12} \\ \Sigma_{21} & \Sigma_{22} \end{pmatrix}$$

$p(y_1) = \mathcal{N}(y_1 \mid \mu_1, \Sigma_{11})$
first marginal

$p(y_2) = \mathcal{N}(y_2 \mid \mu_2, \Sigma_{22})$
second marginal

Gaussian conditioning

$$p(y_1 \mid y_2) = \mathcal{N}\left(y_1 \mid \mu_{1|2}, \Sigma_{1|2}\right)$$

Gaussian conditioning

$$p(y_1 \mid y_2) = \mathcal{N}\left(y_1 \mid \mu_{1|2}, \Sigma_{1|2}\right)$$

$$\mu_{1|2} = \mu_1 + \Sigma_{12}\Sigma_{22}^{-1}(y_2 - \mu_2)$$

Gaussian conditioning

$$p(y_1 \mid y_2) = \mathcal{N}\left(y_1 \mid \mu_{1|2}, \Sigma_{1|2}\right)$$

$$\mu_{1|2} = \mu_1 + \Sigma_{12}\Sigma_{22}^{-1}(y_2 - \mu_2)$$

$$\Sigma_{1|2} = \Sigma_{11} - \Sigma_{12}\Sigma_{22}^{-1}\Sigma_{21}$$

Deriving the conditional: isolate the unknown

$$\tilde{y}_i = y_i - \mu_i, \quad \Lambda = \Sigma^{-1} = \begin{pmatrix} \Lambda_{11} & \Lambda_{12} \\ \Lambda_{21} & \Lambda_{22} \end{pmatrix}$$

- Partition the mean and covariance, then center each block.

Deriving the conditional: isolate the unknown

$$\tilde{y}_i = y_i - \mu_i, \quad \Lambda = \Sigma^{-1} = \begin{pmatrix} \Lambda_{11} & \Lambda_{12} \\ \Lambda_{21} & \Lambda_{22} \end{pmatrix}$$

$$p(y_1 \mid y_2) \propto p(y_1, y_2) \propto \exp\left(-\tfrac{1}{2} q(y_1; y_2)\right)$$

- Partition the mean and covariance, then center each block.
- Conditioning fixes y_2, so factors independent of y_1 become part of the normalizing constant.

Deriving the conditional: isolate the unknown

$$\tilde{y}_i = y_i - \mu_i, \quad \Lambda = \Sigma^{-1} = \begin{pmatrix} \Lambda_{11} & \Lambda_{12} \\ \Lambda_{21} & \Lambda_{22} \end{pmatrix}$$

$$p(y_1 \mid y_2) \propto p(y_1, y_2) \propto \exp\left(-\tfrac{1}{2} q(y_1; y_2)\right)$$

$$q(y_1; y_2) = \tilde{y}_1^T \Lambda_{11} \tilde{y}_1 + 2\tilde{y}_1^T \Lambda_{12} \tilde{y}_2$$

- Partition the mean and covariance, then center each block.
- Conditioning fixes y_2, so factors independent of y_1 become part of the normalizing constant.
- What remains is a quadratic kernel in the unknown block y_1.

Deriving the conditional: complete the square

$$q(y_1; y_2) = \tilde{y}_1^T \Lambda_{11} \tilde{y}_1 + 2\tilde{y}_1^T \Lambda_{12} \tilde{y}_2$$

- The quadratic from the previous slide is a Gaussian kernel waiting to be completed.

Deriving the conditional: complete the square

$$q(y_1; y_2) = \tilde{y}_1^T \Lambda_{11} \tilde{y}_1 + 2\tilde{y}_1^T \Lambda_{12} \tilde{y}_2$$

$$m_{1|2} = -\Lambda_{11}^{-1} \Lambda_{12} \tilde{y}_2$$

- The quadratic from the previous slide is a Gaussian kernel waiting to be completed.
- Choose $m_{1|2}$ so its cross term matches the joint precision cross term.

Deriving the conditional: complete the square

$$q(y_1; y_2) = \tilde{y}_1^T \Lambda_{11} \tilde{y}_1 + 2\tilde{y}_1^T \Lambda_{12} \tilde{y}_2$$

$$m_{1|2} = -\Lambda_{11}^{-1} \Lambda_{12} \tilde{y}_2$$

$$q(y_1; y_2) = \left(\tilde{y}_1 - m_{1|2}\right)^T \Lambda_{11} \left(\tilde{y}_1 - m_{1|2}\right) - m_{1|2}^T \Lambda_{11} m_{1|2}$$

- The quadratic from the previous slide is a Gaussian kernel waiting to be completed.
- Choose $m_{1|2}$ so its cross term matches the joint precision cross term.
- The term without y_1 joins the normalizer; then read off the mean and covariance.

Deriving the conditional: complete the square

$$q(y_1; y_2) = \tilde{y}_1^T \Lambda_{11} \tilde{y}_1 + 2\tilde{y}_1^T \Lambda_{12} \tilde{y}_2$$

$$m_{1|2} = -\Lambda_{11}^{-1} \Lambda_{12} \tilde{y}_2$$

$$q(y_1; y_2) = \left(\tilde{y}_1 - m_{1|2}\right)^T \Lambda_{11} \left(\tilde{y}_1 - m_{1|2}\right) - m_{1|2}^T \Lambda_{11} m_{1|2}$$

$$p(y_1 \mid y_2) = \mathcal{N}(y_1 \mid \mu_1 - \Lambda_{11}^{-1} \Lambda_{12} (y_2 - \mu_2), \Lambda_{11}^{-1})$$

- The quadratic from the previous slide is a Gaussian kernel waiting to be completed.
- Choose $m_{1|2}$ so its cross term matches the joint precision cross term.
- The term without y_1 joins the normalizer; then read off the mean and covariance.

Deriving the conditional: recover covariance blocks

$$p(y_1 \mid y_2) = \mathcal{N}(y_1 \mid \mu_1 - \Lambda_{11}^{-1}\Lambda_{12}(y_2 - \mu_2), \Lambda_{11}^{-1})$$

- Start from the precision-form conditional just derived.

Deriving the conditional: recover covariance blocks

$$p(y_1 \mid y_2) = \mathcal{N}(y_1 \mid \mu_1 - \Lambda_{11}^{-1}\Lambda_{12}(y_2 - \mu_2), \Lambda_{11}^{-1})$$

$$\begin{pmatrix}\Lambda_{11} & \Lambda_{12}\end{pmatrix}\begin{pmatrix}\Sigma_{11} & \Sigma_{12} \\ \Sigma_{21} & \Sigma_{22}\end{pmatrix} = \begin{pmatrix}I & 0\end{pmatrix}$$

- Start from the precision-form conditional just derived.
- The first block row of $\Lambda\Sigma = I$ gives the two identities we need.

Deriving the conditional: recover covariance blocks

$$p(y_1 \mid y_2) = \mathcal{N}(y_1 \mid \mu_1 - \Lambda_{11}^{-1}\Lambda_{12}(y_2 - \mu_2), \Lambda_{11}^{-1})$$

$$\begin{pmatrix}\Lambda_{11} & \Lambda_{12}\end{pmatrix}\begin{pmatrix}\Sigma_{11} & \Sigma_{12} \\ \Sigma_{21} & \Sigma_{22}\end{pmatrix} = \begin{pmatrix}I & 0\end{pmatrix}$$

$$\Lambda_{11}\Sigma_{12} + \Lambda_{12}\Sigma_{22} = 0, \quad \Lambda_{11}\Sigma_{11} + \Lambda_{12}\Sigma_{21} = I$$

- Start from the precision-form conditional just derived.
- The first block row of $\Lambda\Sigma = I$ gives the two identities we need.

Deriving the conditional: recover covariance blocks

$$p(y_1 \mid y_2) = \mathcal{N}(y_1 \mid \mu_1 - \Lambda_{11}^{-1}\Lambda_{12}(y_2 - \mu_2), \Lambda_{11}^{-1})$$

$$\begin{pmatrix}\Lambda_{11} & \Lambda_{12}\end{pmatrix}\begin{pmatrix}\Sigma_{11} & \Sigma_{12} \\ \Sigma_{21} & \Sigma_{22}\end{pmatrix} = \begin{pmatrix}I & 0\end{pmatrix}$$

$$\Lambda_{11}\Sigma_{12} + \Lambda_{12}\Sigma_{22} = 0, \quad \Lambda_{11}\Sigma_{11} + \Lambda_{12}\Sigma_{21} = I$$

$$\Lambda_{11}^{-1} = \Sigma_{11} - \Sigma_{12}\Sigma_{22}^{-1}\Sigma_{21}, \quad -\Lambda_{11}^{-1}\Lambda_{12} = \Sigma_{12}\Sigma_{22}^{-1}$$

- Start from the precision-form conditional just derived.
- The first block row of $\Lambda\Sigma = I$ gives the two identities we need.
- Its Schur complement is the variation left after accounting for y_2; substitute to recover the block formula.

Deriving the conditional: recover covariance blocks

$$p(y_1 \mid y_2) = \mathcal{N}(y_1 \mid \mu_1 - \Lambda_{11}^{-1}\Lambda_{12}(y_2 - \mu_2), \Lambda_{11}^{-1})$$

$$\begin{pmatrix}\Lambda_{11} & \Lambda_{12}\end{pmatrix}\begin{pmatrix}\Sigma_{11} & \Sigma_{12} \\ \Sigma_{21} & \Sigma_{22}\end{pmatrix} = \begin{pmatrix}I & 0\end{pmatrix}$$

$$\Lambda_{11}\Sigma_{12} + \Lambda_{12}\Sigma_{22} = 0, \quad \Lambda_{11}\Sigma_{11} + \Lambda_{12}\Sigma_{21} = I$$

$$\Lambda_{11}^{-1} = \Sigma_{11} - \Sigma_{12}\Sigma_{22}^{-1}\Sigma_{21}, \quad -\Lambda_{11}^{-1}\Lambda_{12} = \Sigma_{12}\Sigma_{22}^{-1}$$

$$p(y_1 \mid y_2) = \mathcal{N}(y_1 \mid \mu_1 + \Sigma_{12}\Sigma_{22}^{-1}(y_2 - \mu_2), \Sigma_{11} - \Sigma_{12}\Sigma_{22}^{-1}\Sigma_{21})$$

- Start from the precision-form conditional just derived.
- The first block row of $\Lambda\Sigma = I$ gives the two identities we need.
- Its Schur complement is the variation left after accounting for y_2; substitute to recover the block formula.

Bivariate conditional

$$p(y_1 \mid y_2) = \mathcal{N}\left(y_1 \mid \mu_1 + \rho\frac{\sigma_1}{\sigma_2}(y_2 - \mu_2), \sigma_1^2(1-\rho^2)\right)$$

$y_2 - \mu_2$
observation

→

$\rho\frac{\sigma_1}{\sigma_2}$
mean shift

→

$\sigma_1^2(1-\rho^2)$
remaining variance

Conditioning as an inference machine

$p(y_1, y_2)$
joint Gaussian

→

y_2
observed

→

$p(y_1 \mid y_2)$
Gaussian posterior

$$\Sigma_{1|2} \leq \Sigma_{11}$$

Summary: Multivariate Gaussian and conditioning

- μ: location; Σ: scale, orientation, and dependence
- Mahalanobis distance follows covariance geometry
- High-dimensional Gaussian mass concentrates in a shell
- Marginals retain covariance blocks
- Conditionals are Gaussian with shifted mean and reduced covariance

Section 10

Missing values as Gaussian conditioning

Inference about hidden entries by conditioning

Visible and hidden entries

$y = (y_v, y_h)$
vector

→

y_v
visible

→

$p(y_h \mid y_v, \theta)$
hidden entries

$$p(y_h \mid y_v, \theta)$$

Summary: Missing values as conditioning

- Partition a vector into visible entries y_v and hidden entries y_h
- Infer hidden entries with $p(y_h \mid y_v, \theta)$
- Gaussian conditioning supplies this distribution analytically

Section 11

Linear Gaussian systems

Latent state, observation, and Gaussian posterior

Linear Gaussian model

$z \sim \mathcal{N}(\mu_z, \Sigma_z)$
latent state

$$p(z, y) = p(z)p(y|z)$$

Linear Gaussian model

$z \sim \mathcal{N}(\mu_z, \Sigma_z)$
latent state

→

$y \mid z \sim \mathcal{N}(Wz + b, \Sigma_y)$
observation

$$p(z, y) = p(z)p(y|z)$$

Joint Gaussian form

$$\mu = \begin{pmatrix} \mu_z \\ W\mu_z + b \end{pmatrix}$$

$$\Sigma = \begin{pmatrix} \Sigma_z & \Sigma_z W^T \\ W\Sigma_z & \Sigma_y + W\Sigma_z W^T \end{pmatrix}$$

$$\begin{pmatrix} z \\ y \end{pmatrix} \sim \mathcal{N}(\mu, \Sigma)$$

Posterior state

$$p(z \mid y) = \mathcal{N}\left(z \mid \mu_{z|y}, \Sigma_{z|y}\right)$$

Posterior state

$$p(z \mid y) = \mathcal{N}\left(z \mid \mu_{z|y}, \Sigma_{z|y}\right)$$

$$\Lambda_{z|y} = \Sigma_z^{-1} + W^T \Sigma_y^{-1} W$$

Posterior state

$$p(z \mid y) = \mathcal{N}\left(z \mid \mu_{z|y}, \Sigma_{z|y}\right)$$

$$\Lambda_{z|y} = \Sigma_z^{-1} + W^T \Sigma_y^{-1} W$$

$$\Sigma_{z|y} = \Lambda_{z|y}^{-1}$$

Posterior state

$$p(z \mid y) = \mathcal{N}\left(z \mid \mu_{z|y}, \Sigma_{z|y}\right)$$

$$\Lambda_{z|y} = \Sigma_z^{-1} + W^T \Sigma_y^{-1} W$$

$$\Sigma_{z|y} = \Lambda_{z|y}^{-1}$$

$$\mu_{z|y} = \Sigma_{z|y}\left(W^T \Sigma_y^{-1}(y - b) + \Sigma_z^{-1}\mu_z\right)$$

Sensor fusion

Prior
$$\mathcal{N}(z \mid \mu_z, \Sigma_z)$$

$\times$

Measurement
$$\mathcal{N}(y \mid Wz + b, \Sigma_y)$$

$\propto$

Posterior
$$\mathcal{N}\left(z \mid \mu_{z|y}, \Sigma_{z|y}\right)$$

$$\Sigma_{z|y}^{-1} = \Sigma_z^{-1} + W^T \Sigma_y^{-1} W$$

Summary: Linear Gaussian systems

- Prior: $p(z) = \mathcal{N}(z \mid \mu_z, \Sigma_z)$
- Observation: $p(y \mid z) = \mathcal{N}\big(y \mid Wz + b, \Sigma_y\big)$
- Joint model: $p(z, y) = p(z)p(y|z)$
- Posterior: $p(z|y)$ is Gaussian
- Precision adds information from the prior and observation

Lecture 1 summary

- ML tasks specify observed quantities, unknown quantities, and required outputs
- Moments and conditional moments describe probabilistic summaries
- Categorical, multinomial, and softmax distributions describe labels
- Covariance gives multivariate geometry
- Gaussian marginalization and conditioning are reusable inference machinery
- Linear Gaussian systems provide the first complete inference machine